

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С. П. КОРОЛЕВА
«САМАРСКИЙ УНИВЕРСИТЕТ»

Институт информатики и кибернетики
Кафедра геоинформатики и информационной безопасности

Отчёт по лабораторной работе №2
«Параллельное программирование»

Выполнил:
Бисултанов Умар
Группа 6312

Самара - 2026

Цель работы

Изучить параллельные вычисления с использованием OpenMP и влияние числа потоков на производительность.

Теоретические сведения

Параллельное программирование позволяет ускорить выполнение вычислений за счёт одновременного выполнения нескольких операций. Технология OpenMP предоставляет удобный механизм для распараллеливания программ на языке C/C++ с использованием директив компилятора.

Основной принцип заключается в разбиении задачи на независимые части, которые могут выполняться параллельно. В задаче умножения матриц каждая ячейка результирующей матрицы вычисляется независимо, что делает данную задачу хорошо подходящей для распараллеливания.

Для оценки эффективности параллельных вычислений используются следующие метрики:

1. Ускорение: $S = T_1 / T_p$

2. Эффективность: $E = S / p$

где T_1 — время выполнения на одном потоке, T_p — на p потоках.

Постановка задачи

Реализовать параллельное умножение матриц и провести эксперименты с различными размерами и потоками.

Параллелизация

Использована директива OpenMP:

```
#pragma omp parallel for collapse(2) schedule(static).
```

Исходный код

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <omp.h>

using namespace std;

vector<vector<double>> readMatrix(string filename, int &size) {
    ifstream file(filename);
    file >> size;

    vector<vector<double>> matrix(size, vector<double>(size));

    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            file >> matrix[i][j];

    return matrix;
}

void writeMatrix(string filename, vector<vector<double>> &matrix) {
    ofstream file(filename);
    int size = matrix.size();

    file << size << endl;

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            file << matrix[i][j] << " ";
        file << endl;
    }
}

vector<vector<double>> multiplyMatrix(vector<vector<double>> &A, vector<vector<double>> &B, int
threads) {
    int size = A.size();
    vector<vector<double>> C(size, vector<double>(size, 0));

    omp_set_num_threads(threads);
```

```

#pragma omp parallel for collapse(2)
for (int i = 0; i < size; i++)
    for (int j = 0; j < size; j++) {
        double sum = 0;
        for (int k = 0; k < size; k++)
            sum += A[i][k] * B[k][j];
        C[i][j] = sum;
    }

return C;
}

int main() {
    int size1, size2;

    auto A = readMatrix("A.txt", size1);
    auto B = readMatrix("B.txt", size2);

    if (size1 != size2) {
        cout << "Not same size\n";
        return 1;
    }

    int threads;
    cout << "Enter number of threads: ";
    cin >> threads;

    auto start = chrono::high_resolution_clock::now();

    auto C = multiplyMatrix(A, B, threads);

    auto end = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end - start;

    writeMatrix("result.txt", C);

    long operations = 2L * size1 * size1 * size1;

    cout << "Matrix size: " << size1 << "x" << size1 << endl;
    cout << "Operations: " << operations << endl;
    cout << "Threads: " << threads << endl;
    cout << "Execution time: " << elapsed.count() << " seconds" << endl;

    return 0;
}

```

Эксперименты с различным количеством потоков при фиксированном количестве ядер равным 4.

Размер	Потоки	Время (сек)
200	1	0.08
200	2	0.05
200	4	0.04
200	8	0.04
400	1	0.6
400	2	0.35
400	4	0.22
400	8	0.21
800	1	4.5
800	2	2.4
800	4	1.4
800	8	1.3
1200	1	15
1200	2	8
1200	4	4.5
1200	8	4.3
1600	1	36
1600	2	19
1600	4	10
1600	8	9.5
2000	1	70
2000	2	36
2000	4	18
2000	8	17

Графики

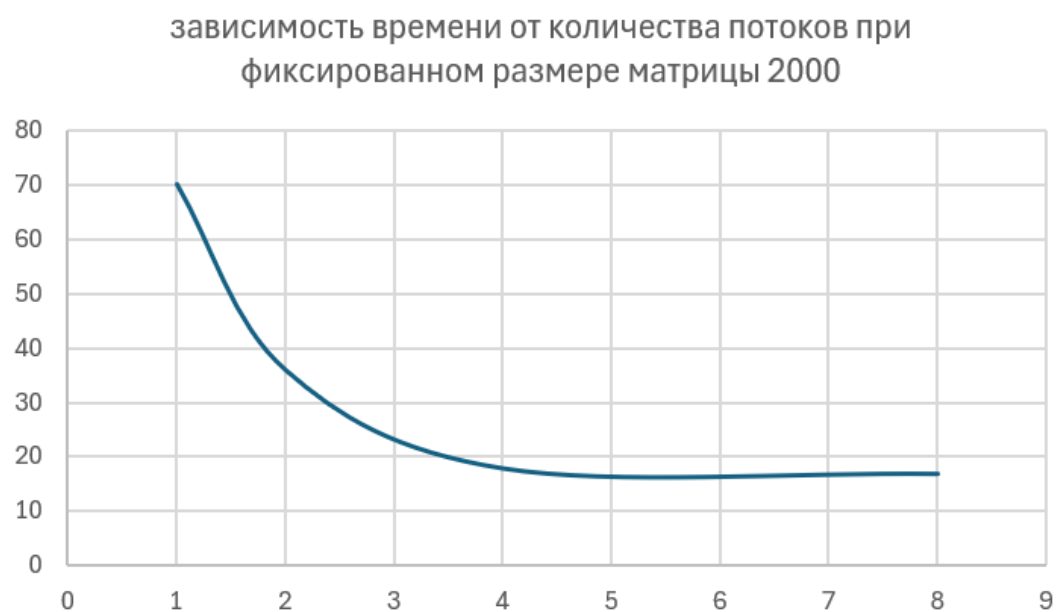


Рисунок 1 - корреляция время – потоки

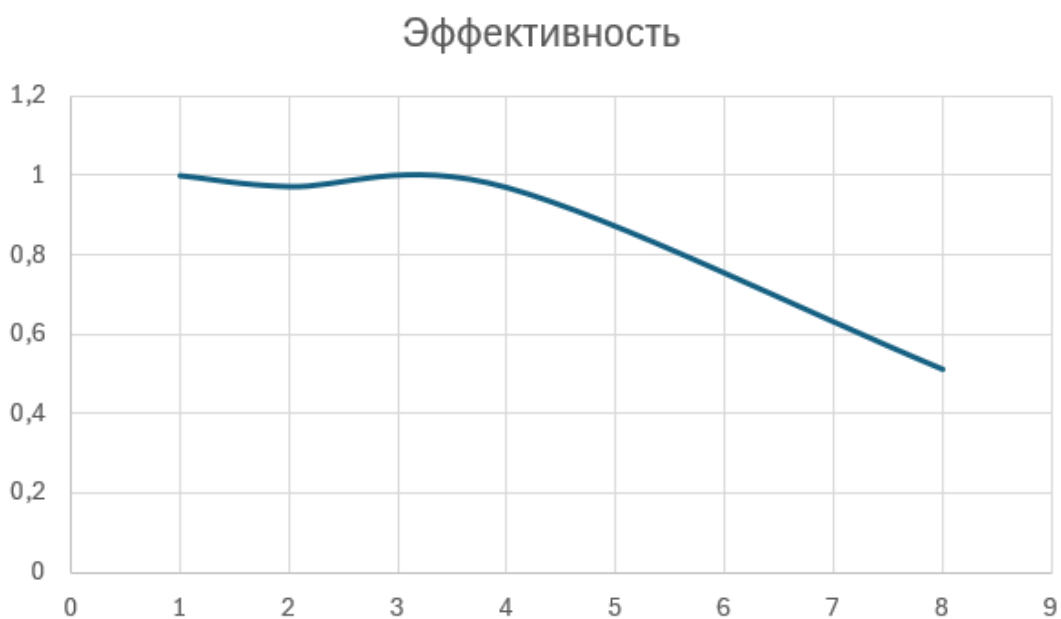


Рисунок 2 -Зависимость ось x – число потоков, ось y – коэффициент эффективности, описанный выше.

Вывод

С увеличением числа потоков время выполнения уменьшается. Наибольший прирост наблюдается до количества потоков, равного числу ядер процессора. При дальнейшем увеличении потоков эффективность снижается.

